

Warcrest — AI 활용 기술 문서

기획부터 구현·디버깅·아트·오디오·배포·문서화까지 전 과정을 AI 코딩 에이전트와의 협업으로 진행한 기록

1. 요약 — 대략 어떻게 활용했는가

이 프로젝트는 2026-07-23 저장소 생성부터 2026-08-10 제출 직전까지 약 19일, **300여 개의 작업** 턴에 걸쳐 **기획 · 설계 · 코드 구현 · 버그 수정 · 아트 파이프라인 · 오디오 · 배포 · 문서화**의 전 과정을 AI 코딩 에이전트와의 대화형 협업으로 만들었습니다. 사람이 직접 짠 코드는 없으며, 사용자는 **방향을 제시하고 결과물을 확인·평가·재지시하는 역할**을 맡았습니다.

사용한 AI 도구는 두 가지입니다.

- Claude Code** — 메인 개발 에이전트. 구조 설계, 대규모 리팩터링, 오디오 시스템, 배포/인프라, 이 문서를 포함한 제출 문서화까지 폭넓게 담당.
- Codex (GPT-5)** — 초기 게임 디자인 협상(스킬/레벨/AI 행동 등 세부 규칙을 50여 턴의 1문1답으로 확정)과, 8월 초 유닛 스프라이트·애니메이션 파이프라인처럼 좁고 반복적인 자산 QA 작업에 집중 투입.

두 에이전트는 **같은 저장소를 번갈아, 때로는 같은 날 같은 기능을 이어받아** 작업했습니다(4장 참고). 이를 안전하게 가능케 한 것이 아래 2장의 **"하네스" (에이전트 운영 규약) 체계**입니다.

2. 활용 방식 상세 — 하네스 기반 협업 체계

2.1 왜 하네스가 필요했는가

한 세션이 끝나면 그 에이전트의 문맥(대화 기억)은 사라집니다. 이 프로젝트는 여러 날에 걸쳐 여러 세션(때로는 Claude Code와 Codex가 같은 날 번갈아)이 같은 저장소를 계속 이어받아 작업해야 했기 때문에, **"이 저장소는 지금 어떤 규칙으로 운영되고, 무엇을 했고, 다음에 뭘 해야 하는지"**를 **코드 저장소 자체에 기록해 두는 방식**이 필요했습니다. 이것이 저장소 루트의 `AGENTS.md` 를 시작점으로 하는 하네스입니다.

2.2 핵심 규칙

구성 요소	역할
<code>AGENTS.md</code>	모든 에이전트가 세션 시작 시 가장 먼저 읽는 최상위 규약. "The One Rule" — 변경한 내용은 반드시 백로그 기록(GitHub Issue 또는 <code>docs/dev-wiki/backlog.md / log.md</code>)을 남겨야 한다는 단 하나의 강제 규칙을 명시.
<code>docs/dev-wiki/log.md</code>	append-only 작업 이력. 무엇을, 왜, 어떻게 검증했는지 매 턴 기록 — 다음 세션(같은 에이전트든 다른 에이전트든)이 여기서부터 이어갈 수 있게 함.
<code>docs/dev-wiki/backlog.md</code>	진행 중/예정 작업 큐. 우선순위가 높은 미해결 이슈(예: 유닛 반투명 버그)를 다음 세션이 놓치지 않도록 최상단에 고정.
<code>docs/ai-usage/session-log.md</code>	이 대회 제출을 위한 별도 트랙 — 무엇을 했는지가 아니라 "AI에게 어떻게 지시했는지"를 기록. 방향 전환/피드백 턴은 요약이 아니라 사용자 원문을 그대로 인용 하도록 규칙화되어 있음(제출물 4번이 "AI 대상 주요 프롬프트 및 지시 사항"을 명시적으로 요구하기 때문).

2.3 검증 위주 작업 습관

하네스는 기록뿐 아니라 **"자기 보고를 그대로 믿지 않고 직접 확인한다"**는 작업 습관도 반복적으로 만들어냈습니다. 예를 들어 GitHub Pages 배포 문제는 "코드가 문제일 것"이라고 추측하는 대신 GitHub REST API로 실제 워크플로 실행 기록을 조회해 원인을 특정했고, 효과음이 원본과 실제로 다른지는 "다를 것"이라고 서술하는 대신 두 파형의 상관계수를 numpy로 직접 계산해 수치(-0.23, 0.005 등)로 제시했습니다. Playwright를 이용한 실제 화면 스크린샷 검증도 거의 모든 UI 변경에 동반되었습니다.

3. 게임 컨셉·디자인·플레이 반복 개발 스토리

이 프로젝트에서 가장 두드러진 패턴은, 사용자가 큰 그림/느낌을 제시하면 AI가 구현하고, 사용자가 실제로 플레이해보고 평가한 뒤 재지시하는 사이클이 처음부터 끝까지 반복됐다는 점입니다. 세부 수치나 코드 구조를 사용자가 미리 정해준 적은 거의 없고, 대신 결과물을 보고 "이게 아니다"/"이거 맞다"를 판정하는 방식으로 방향이 다져졌습니다.

3.1 1차 컨셉과 첫 번째 방향 정정

최초 컨셉은 "디아블로 던전을 실제로 걸어 다니며 적과 부딪혀 싸우고, 갠된 동료를 구출해 대열을 늘리는" 실시간 액션이었습니다. 그런데 첫 구현은 이를 "왼쪽 길 / 오른쪽 길"을 고르는 메뉴 버튼형 로그라이트로 추상화해버렸고, 사용자는 이를 즉시 되돌렸습니다.

사용자 원문 (2026-07-23)

"디아블로 던전에서 실제로 움직이며 적을 죽이거나 감옥/골방에 묶인 동료를 구출"

AI는 메뉴 기반 버전을 폐기하고 WASD 실시간 이동 + 절차적 던전 생성 + BFS 자동 경로탐색 기반 헤드리스 플레이테스트로 전면 재구현했습니다.

3.2 비주얼 톤 정정 — 2단계

사용자 원문

"디자인 너무 허접함. 80년대 게임 같음. 적어도 2000년대 초반 정도의 3D홍내내는 것과 같은 모습"

1차 대응(하이라이트/그림자/아웃라인 추가)으로도 부족하다는 두 번째 피드백이 이어졌습니다.

사용자 원문

"입체감 자체가 잘못되었다... 디아블로 같은 등각 투영... 클래시 오브 클랜·클래시 로얄처럼"

이번엔 추상적 지적이 아니라 등각(isometric) 투영 / 톤(밝고 채도 높은 클래시 오브 클랜풍) / 병력 무리 이동 세 가지로 구체적으로 재지시했고, AI는 진짜 3D 렌더링은 이 환경에서 불가능함을 먼저 사용자에게 알리고 확인받은 뒤, 좌표계 전체를 등각 투영으로 재작성했습니다.

3.3 전투 시스템 정정 — 파타풍식 → MMO 핫바식

당초 "정해진 순서를 입력하고 공용 타이머 안에 처리"하는 파타풍류 커맨드 전투로 구현했으나, 사용자가 원하는 것은 "MMORPG처럼 슬롯에 미리 배치해두고 쿨타임이 끝날 때마다 적재적소에 누르는" 감각이었습니다. 전투 시스템 (combat.ts)을 각 슬롯이 독립 쿨타임을 갖는 방식으로 전면 재작성했습니다.

3.4 세부 규칙 협상 — Codex와의 50여 턴 1문1답

스킬 슬롯 수, 레벨업 포인트, 마나/포션, 인벤토리, 적 AI 행동(배회/추격/도주) 같은 세부 규칙은 Codex가 매번 두세 개의 선택지를 제시하고 사용자가 "1", "2번", "중간 수준" 같은 한두 단어로 답하는 방식으로 약 50턴에 걸쳐 확정됐습니다. 이 과정에서 AI가 사용자 의도를 잘못 이해해 되짚어야 했던 순간도 있었습니다.

사용자 원문

"아니 무슨 소리야. 뭔가 잘못 이해하고 있는 거 같은데?"

3.5 "재미없다" 피드백 → 스프라이트 전면 교체

도형 기반 캐릭터로 필드 자동전투까지 구현한 뒤, 사용자가 "노잼"이라고 평가하자 AI는 절차적 도형을 생성형 이미지 스프라이트로 전면 교체했습니다. 교체 직후 "제대로 적용이 안 된 것 같다"(캐릭터/적이 비정상적으로 거대해 보임)는 재지적이 있었고, 원인을 추적한 결과 바운스 애니메이션이 setScale 을 초기화하지 않는 버그였음을 찾아 수정했습니다.

3.6 최종 대전환 — 던전 액션에서 레인 공성 경제 게임으로

구조/전투/스킬을 거의 다 완성한 시점에, 사용자는 게임을 통째로 다른 장르로 바꾸기로 결정했습니다.

사용자 원문 (2026-07-25)

"게임 컨셉을 조금 바꾸자. 내가 원하는 게 뭔지 정확히 깨달았어." / "워크래프트3 기반의 civilization wars라는 커스텀 게임이 있어..."

AI는 LoL/DotA(레인·본진·자동 생성 미니언 구조)와 Civilization Wars(시대별 기술 해금, 자원 기반 인구 배치)를 리서치해 `concept-pivot-lane-siege.md` 전환 설계 문서를 작성했고, 이를 근거로 자원/시대/일꾼/웨이브/거점 시스템을 새로 설계했습니다. 이것이 지금의 **Warcrest**입니다. 기존 프로토타입에서는 등각 렌더링 파이프라인, 모바일 친화 입력, HUD 톤만 계승하고 나머지(던전 생성, 구출 루프, 인벤토리 등)는 폐기했습니다.

전환 이후에도 지형/카메라 프로토타입 여러 판, 2라인 맵 구조, HUD 레이아웃 등이 스크린샷 기반 검토 → 재지시 → 재구현 사이클을 계속 거쳤습니다.

사용자 원문

"일단 UI가 너무 잘못됨."

4. 대표 사례로 보는 "제시 → 구현 → 검토 → 재작업" 반복

가장 반복 횟수가 많고 근본 원인 진단까지 필요했던 사례를 상세히 남깁니다 — 전투 효과음(SFX) 재작업입니다.

Claude Code 전투 효과음 v1 → v4, 4단계 재작업

v1 (순수 절차적 합성) — Web Audio 오실레이터/노이즈로 타격음을 그 자리에서 합성. 사용자: "아 뭐야 음악이 여전히 기계음 같잖아 진짜 엉망이네. 다시 음악 제대로 생성해줘"

v2 (실제 CC0 녹음 사용) — Kenney.nl의 CC0 사운드팩에서 실제 녹음(종 타격, 폭발음 등)을 잘라 붙이는 방식으로 전환.

v3 (분석 기반 재합성 요구) — 사용자: "완전 무료/저작권 없음도 문제가 있을 수 있음. 해당 음을 기반으로 네가 직접 재 생성해줘. 똑같은 안 된다고 생각하면 되." CC0라도 원본 파형을 그대로 쓰는 것 자체가 부담이라는 지적 → 원본을 분석(공명 주파수·스펙트럼 추이)해 사인파로 재합성하는 방식으로 변경.

재지적 — 사용자: "뭐가 바뀐 건지 전혀 모르겠어... 정말 개선이 하나도 안 된 거 같아" AI가 재점검한 결과, "분석"을 했어도 합성 수단이 여전히 깨끗한 사인파라면 v1과 똑같이 "기계음"으로 들릴 수밖에 없다는 근본 원인을 발견.

v4 (그레인러 재합성) — 순음 오실레이터를 완전히 배제하고, 실제 녹음을 5~20ms 그레인으로 쪼개 위치를 지터링하며 재조립하는 방식으로 교체. 원본 파형과의 상관계수를 직접 계산(-0.005~0.02, 사실상 무관)해 "동일하지 않음"을 수치로 검증하면서, 동시에 실제 녹음의 질감은 보존.

Claude Code GitHub Pages 배포 점검

사용자가 README 작성과 "GitHub Pages가 뭐냐"는 질문을 하는 과정에서, AI가 "문제없을 것"이라 가정하지 않고 GitHub REST API로 실제 배포 상태를 조회 → 저장소가 비공개이고 Pages가 한 번도 활성화된 적이 없어 배포 워크플로가 3번 다 실패하고 자동 비활성화되어 있음을 발견. 사용자에게 공개 전환/재활성화 여부를 명시적으로 확인 받은 뒤 조치, 실제 라이브 URL이 200 OK로 응답하는 것까지 검증.

Codex + Claude Code 같은 날 동일 기능 이어받기

2026-08-08, 사용자가 HUD 색상 버그(자원이 부족한데도 지을 수 있는 것처럼 파란색으로 표시됨)를 지적하자 Codex가 원인을 코드에서 직접 확인 (전략 버튼에는 `setStrategicActionEnabled()` 가 있지만 거점/타워 액션에는 대응하는 활성화 로직이 없었음)한 뒤 수정을 진행했고, 같은 날 다른 시간대에는 Claude Code가 오디오 시스템 작업을 이어받았습니다. 두 에이전트 모두 이전 세션의 작업 내용을 추측하지 않고 현재 코드/하네스 문서를 다시 읽고 시작하는 방식으로 충돌 없이 협업했습니다.

5. 외부 에셋 · 오픈소스 출처

자산	출처	라이선스	사용 방식
----	----	------	-------

전투 효과음 참조 녹음 (Impact Sounds, Sci-fi Sounds, RPG Audio)	Kenney.nl	CC0 (Public Domain)	원본 파형을 직접 출력에 포함하지 않음 — 스펙트럼 분석 후 그래놀러 재합성으로 완전히 새로운 파형을 생성(<code>tools/audio-synth/</code>). 원본 라이선스 파일은 <code>tools/audio-synth/vendor/</code> 에 함께 보관.
배경음악, 캐릭터/건물/지형 스프라이트, HUD 아트	자체 생성	해당 없음(자체 제작)	AI 이미지/오디오 생성 도구로 직접 제작.
Phaser 3, TypeScript, Vite	오픈소스 라이브러리	각 프로젝트 OSS 라이선스	게임 엔진/빌드 툴체인으로 사용(코드/에셋이 아닌 프레임워크).

6. 마무리

이 문서에 기록된 협업 패턴을 요약하면: 사용자는 스크린샷과 실제 플레이 경험을 근거로 방향을 제시·평가했고, AI는 그때마다 표면적 증상을 임시 봉합하기보다 근본 원인을 코드/데이터로 직접 확인한 뒤 수정했으며, 모든 변경은 하네스 규약에 따라 저장소에 기록되어 다음 세션(같은 에이전트든 다른 에이전트든)이 그대로 이어받을 수 있었습니다. Claude Code와 Codex를 용도에 맞게 병행한 것도 같은 원칙의 연장선입니다 — 좁고 반복적인 자산 QA는 Codex의 촘촘한 확인-루프에, 구조적 설계와 인프라/문서화처럼 넓은 맥락을 유지해야 하는 작업은 Claude Code에 맡겼습니다.

7. 근거 문서 지도

이 문서는 요약본입니다. 심사자가 실제 턴 단위 기록을 직접 확인하고 싶다면 아래 문서가 저장소에 그대로 공개되어 있습니다.

문서	내용
<code>docs/ai-usage/session-log.md</code>	이 문서의 원본 근거. 2026-07-23 ~ 08-10, 약 300여 개 턴의 지시/작업/검증 기록(방향 전환 turn은 사용자 원문 그대로 인용).
<code>docs/dev-wiki/log.md</code>	코드 변경 관점의 전체 작업 이력(append-only) — "무엇이 왜 바뀌었는지" 중심.
<code>docs/dev-wiki/concept-pivot-lane-siege.md</code>	던전 액션 → 레인 공성 경제 게임 장르 전환 시점의 리서치·설계 전문.
<code>docs/dev-wiki/game-concept.md</code>	초기 컨셉과 방향 정정 이력(80년대 느낌 → 등각/클래시 오브 클랜 톤 전환 등).
<code>AGENTS.md</code>	이 저장소의 에이전트 운영 규약("The One Rule") 원문.